

무엇을 측정할 것인가

Mail Agent로 들여다본 데이터 엔지니어링

Pything 10회차 · 김현수

Act 1

Pain Point : 설계자는 무엇을 먼저 묻는가

어느 면접의 한 장면

네이버 면접 Q11~14

"'좋아졌다'는 감상이다. 엔지니어는 지표로 말해야 한다"

- 군 시절 ops 프로젝트에서 팀 생산성을 크게 끌어올렸다고 자신했다
- 그런데 "그 개선을 무슨 지표로 증명할 것인가" 앞에서 막혔다
- 면접관이 파고든 지점 → "올바른 지표를 고르고 설계할 줄 아는가"

이 발표를 관통하는 하나의 질문

"내가 만든 개선을, 무엇으로 증명할 것인가"

이 질문 하나가 세 단계로 풀린다

① 상황을 해석

군대·인턴의 S·T
= 주어진 상황·목표

② 설계로 답한다

Architecture
= NFR을 정하고 시스템 설계

③ 지표로 증명

Result
= 관측·측정으로 SLO 충족 입증

세 개의 다른 이야기가 아니다. S·T → A → R 하나의 사슬이다

STAR에서 진짜 어려운 부분

이 사슬을 이력서 언어로 옮기면 STAR다. 그런데 S와 T는 이미 주어진다

주어지는 것

- Situation : 상황
- Task : 추상적 목표

내가 만드는 것

- Architecture : S/T를 해석한 설계
- Result : 지표로 증명하는 결과

진짜 역량 = S/T를 어떻게 해석해 설계하고, R을 어떻게 증명하느냐

설계자의 첫 질문

"무엇을 만들까(FR)"도 중요하다

그런데 "무엇을 지킬까(NFR)"는 어디서 오는가?

기능은 답이 있는 문제, 비기능은 제약과 상황에 기반하는 문제

기능 vs 비기능 요구사항

기능 요구사항 (FR)

"메일을 요약·분류한다"

- 있으면 되는 것
- 대체로 명세로 주어짐
- 답이 정해진 문제

비기능 요구사항 (NFR)

"얼마나 빠르게? 안정적으로? 싸게?"

- 얼마나 를 정하는 것
- 제약과 상황이 값을 규정
- 상황을 읽어야 보이는 문제

MAU 30만이라는 상황이 P95 60초라는 NFR을 규정한다 (뒤에서 다시)

오늘의 핵심 명제

비기능 요구사항의 우선순위를 정하는 것이 설계자의 핵심 역량이다

그리고 그 우선순위는, 측정 가능해야 한다

Act 2

이론 지도 : 시스템을 보는 다섯 개의 렌즈

오늘의 지도

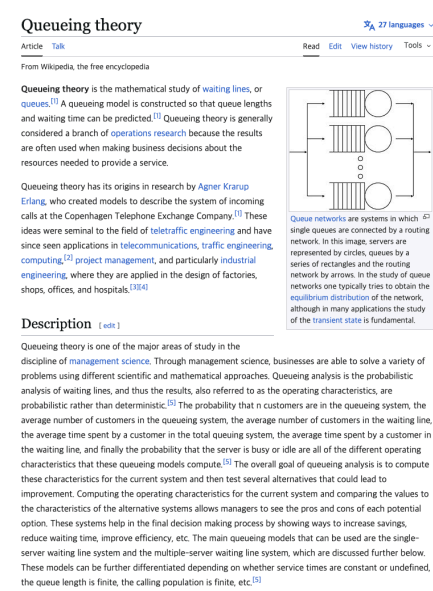
이 발표는 Kafka 튜토리얼이 아니다. 측정 중심으로 시스템을 보는 법

흐름

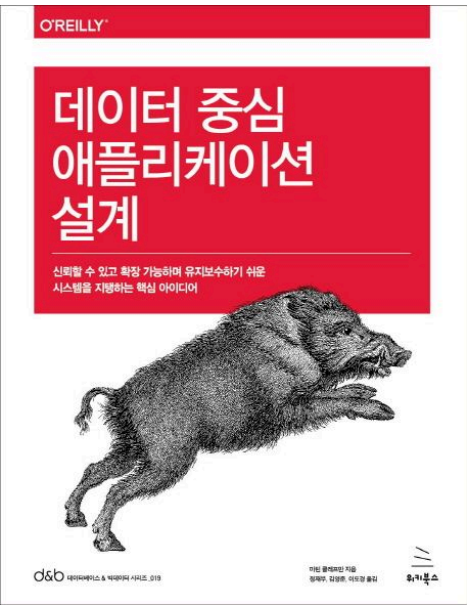
1. Pain : 못 측정한 후회
2. 이론 : 무엇을 볼 수 있나 (5렌즈)
3. Observability : 어떻게 측정하나
4. Mail Agent : 실제로 측정해봤다

세 개의 참고 축

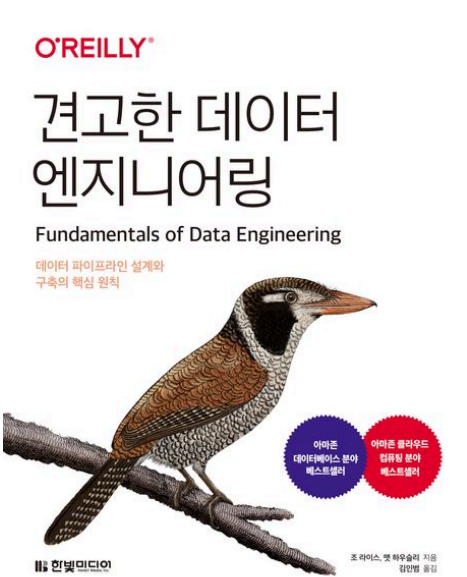
이론을 세 권의 책으로 삼각측량한다. 같은 문제를 서로 다른 언어로



Queueing Theory
성능·지연·용량의 수학



DDIA
분산 데이터의 정석



Fundamentals of DE
수명 주기와 실무

Queueing Theory : 병목을 수식으로 보는 렌즈



오늘의 Hero Tool : Little's Law

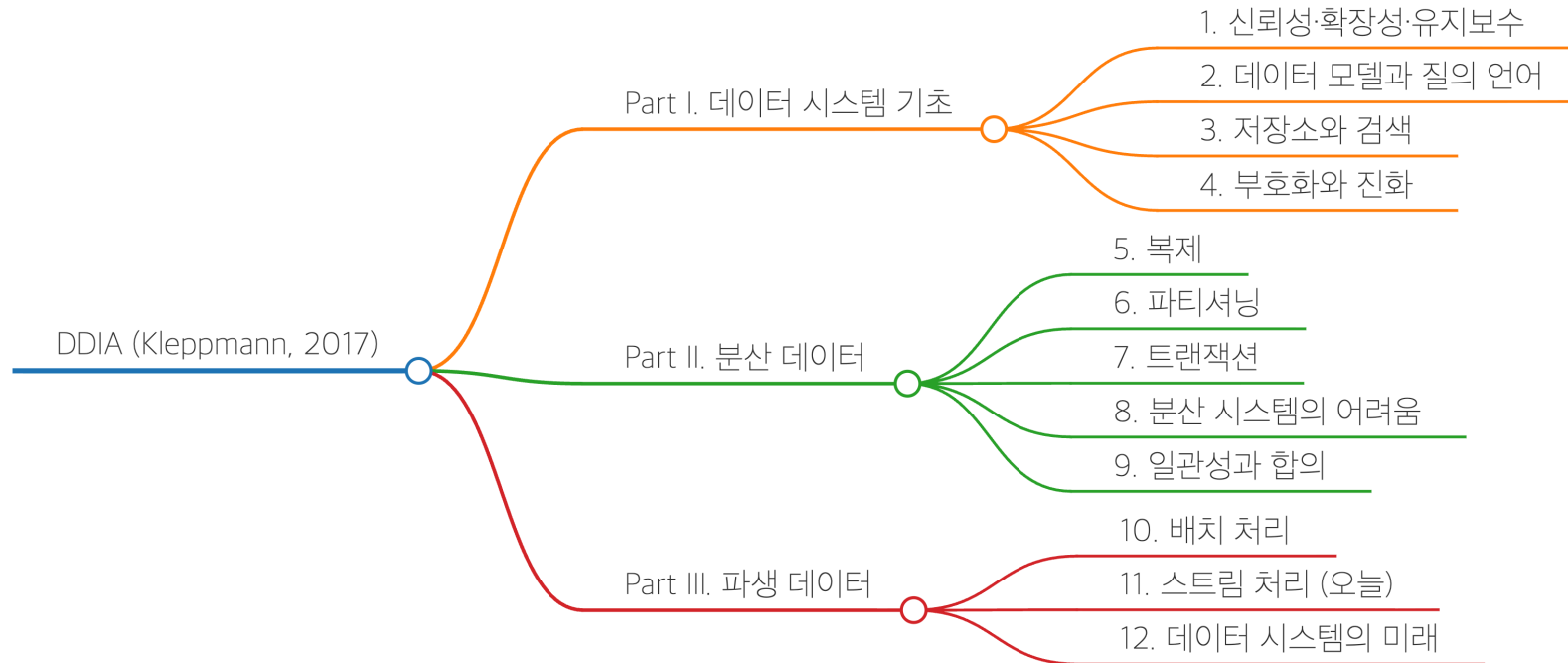
$$L = \lambda \times W$$

- $\underline{L}$ = 시스템 내 평균 요청 수 (대기 포함)
- $\underline{\lambda}$ = 도착률 (초당 유입)
- $\underline{W}$ = 평균 체류 시간

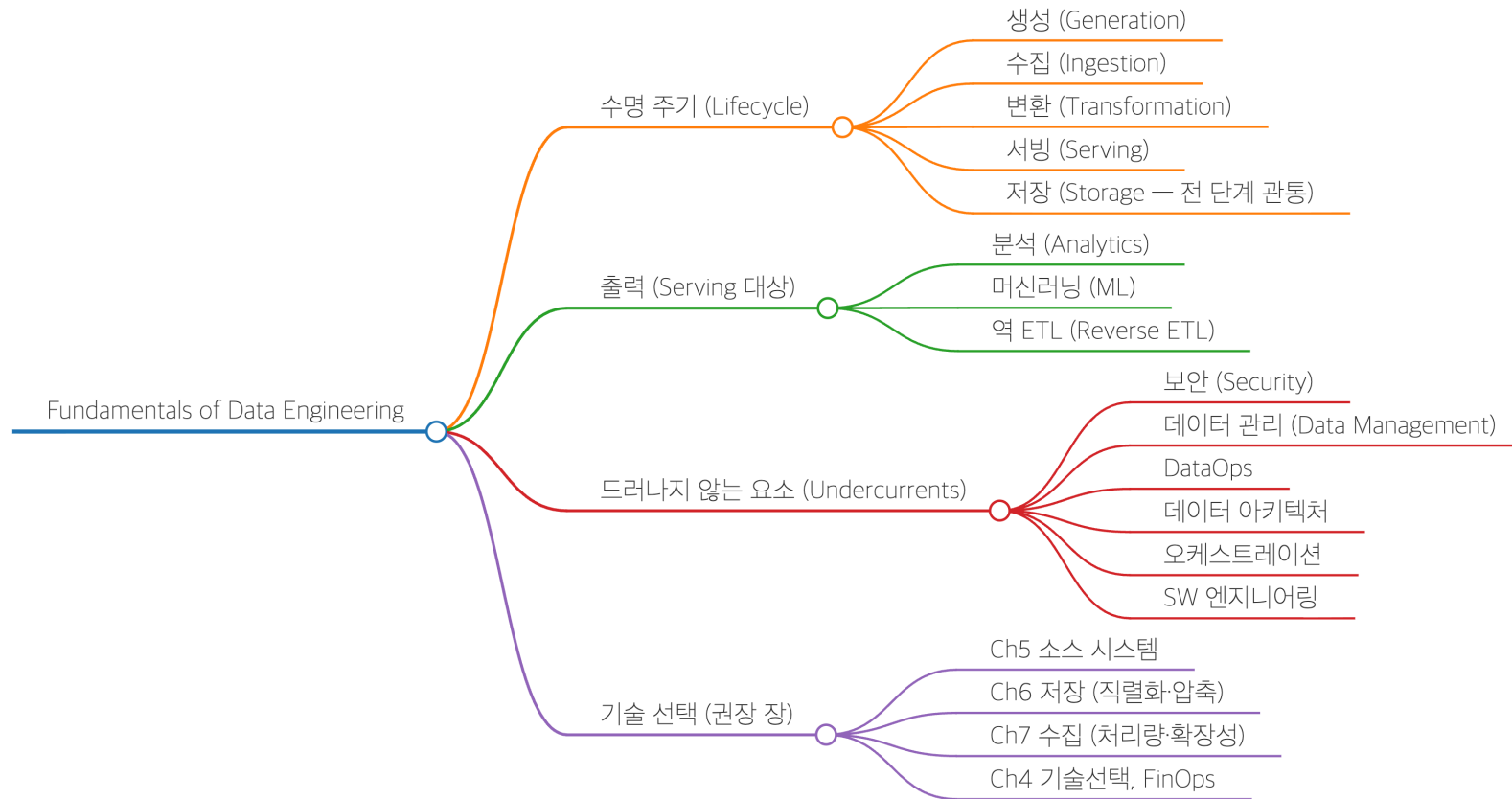
λ 가 그대로인데 $\underline{L}$ 이 계속 늘어난다면 → 처리율(μ)이 못 따라가고 있다는 신호
= 병목 · 큐 적체 · consumer lag

주의: $\underline{W}$ 는 평균 체류시간. P95 같은 꼬리 지연은 별도 렌즈로 본다 (뒤에서)

DDIA : 분산 시스템을 보는 렌즈



Fundamentals : DE 수명주기를 보는 렌즈



측정은 어디에 심나

수명 주기의 마지막 단계가 아니라, 전 구간에 심어야 한다

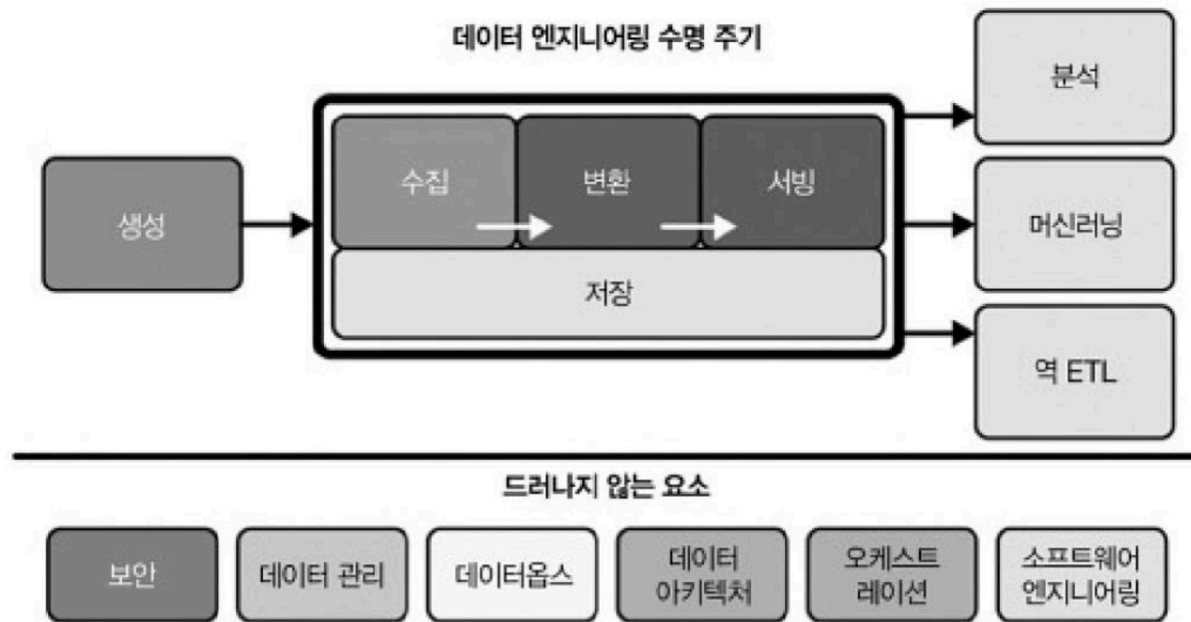


그림 1-1 데이터 엔지니어링 수명 주기

생성 → 수집 → 저장 → 변환 → 서빙, 모든 이음새가 측정 지점

시스템 디자인 5계열

이론을 다섯 개의 렌즈로. 각 계열이 던지는 질문이 다르다

계열	던지는 질문
<u>확률 · 운용과학</u>	병목은 어디인가? 처리율은 충분한가?
<u>분산시스템 이론</u>	장애가 나도 데이터가 남는가?
<u>정보이론 · 부호화</u>	데이터를 안전하게 표현·전송하나?
<u>제어이론</u>	상태를 관측하고 조정할 수 있나?
<u>경제 · 게임이론</u>	비용·운영 부담을 감당할 수 있나?

매핑 슬라이드 (이론 × 이론)

세 권의 책이 같은 문제를 다른 언어로 말한다

계열	Queueing Theory	DDIA	Fundamentals
성능·용량	Little's Law, P95/P99, 병목	Ch1 신뢰성, Ch11 스트림(랙·백프레셔)	Ch7 수집(처리량·확장성)
분산·일관성		Ch5 복제, Ch6 파티셔닝, Ch9 합의	Ch5 소스 시스템
데이터 표현		Ch3 저장, Ch4 부호화, Ch11 로그	Ch6 저장(직렬화·압축)
안정성·피드백	백프레셔, Loss systems	Ch8 분산의 어려움, Ch11 내결함성	Undercurrents(DataOps)
트레이드오프	USL, Queueing Networks	Ch7 트랜잭션, Ch11 AMQP vs 로그	Ch4 기술선택, FinOps

Act 3

Observability : 측정의 인프라

모니터링 ≠ 관측성

모니터링 (Monitoring)

알고 있는 문제를 감시

- 미리 정한 대시보드
- "CPU 80% 넘으면 알림"
- 질문을 미리 알아야 함

관측성 (Observability)

모르는 문제를 사후에 물어봄

- 임의의 질문을 던질 수 있음
- "왜 이 유저만 느렸지?"
- 사후에 파고들 수 있는 능력

MTTR(복구 시간)을 줄이는 유일한 길 = 빨리 원인을 찾는 능력

Observability의 3기둥

Logs

사건을 본다
"무슨 일이 있었나"

Metrics

상태를 본다
"지금 얼마나"

Traces

흐름을 본다
"어디서 시간을 썼나"

처음부터 심어야 하는 이유

- 장애가 난 뒤에는 과거를 측정할 수 없다
- 측정 지점은 설계 시점에 결정해야 한다
- "나중에 붙이자"는 → 정작 필요한 순간 데이터가 없다

무엇을 심을까 : RED / USE

RED (요청 관점)

- Rate : 초당 요청 수
- Errors : 실패율
- Duration : 지연 시간

서비스가 사용자에게 보이는 얼굴

USE (자원 관점)

- Utilization : 사용률
- Saturation : 포화도 (큐 적체)
- Errors : 오류

자원이 버티는지 보는 눈

데이터 파이프라인에서 특히 중요한 건 Saturation = consumer lag, queue depth

시스템 지표를 비즈니스 언어로

DE의 핵심 역량 = 시스템 지표 ↔ 비즈니스 지표 매핑

시스템 지표	비즈니스 임팩트
consumer lag ↑	데이터 신선도 저하 → 의사결정 지연
produce error ↑	이벤트 유실 → 정산·추천 누락
P99 latency ↑	실시간 처리 품질 저하 → 이탈
retention 부족	장애 후 재처리 불가

팀 관점의 표준 답 = DORA 4지표 (배포빈도 · 리드타임 · 변경실패율 · 복구시간)

지표의 함정

측정을 시작하면 새로운 위험이 생긴다

Goodhart's Law

"지표가 목표가 되면 지표이길 멈춘다"
숫자만 맞추는 최적화

꼬리 털림 (Tail)

평균은 좋는데 P95/P99가 터진다
평균 latency의 착시

허영 지표 (Vanity)

보기 좋지만 의사결정에 쓸모없는 숫자
"총 처리 건수 1억!"

평균이 좋아도 꼬리가 터지면 사용자는 느리다 (다음 사례의 핵심)

Act 4

Mail Agent : NFR을 측정 가능한 설계로

Case: Mail Agent

뤼튼테크놀로지스 인턴십에서 구축한 메일 요약·분류 시스템

무엇을

- LangGraph 기반 Agent 워크플로우
- Gmail 연결 → 수집 → LLM 메타데이터 추출 → AI 브리핑
- Kafka pub/sub로 BE ↔ AI 서버 비동기 통신

PM이 준 비기능 요구사항

- MAU 30만 명
- 온보딩 P95 60초 이내
- (LLM 호출 비용도 통제할 것)

NFR을 부하로 번역하기

"MAU 30만"은 비즈니스 언어. Locust에 넣을 수 없다

번역 (Little's Law)

$$L = \lambda \times W$$

- $W \leq \underline{60\text{초}}$ (P95 목표)
- MAU 30만 → DAU 3~6만 (10~20%)
- 피크 계수 3~5배 → 목표 $\lambda \approx 60 \text{ RPS}$

시니어의 첫 일 = 비즈니스 숫자를 $\lambda \cdot W$ 로 번역
이 번역이 안 되면, 부하 테스트에 무엇을 넣을
지도 모른다

파트 A

요구사항만 받고도, 이 설계를 뽑았어야 했다

시니어라면 : 숫자에서 병렬을 역산한다

목표 λ 와 1명 처리시간만 있으면, 필요한 병렬 수가 계산된다

1명 처리시간부터 실측

- 메일 수집 + LLM 요약 + 분류 + 브리핑
- 합치면 1명이 SLO(60초)를 거의 다 씀
- 그래서 동시에 여러 명을 병렬로 돌려야 함

병렬 수 = 목표 처리량 ÷ 한 대 처리율

- 60 RPS를 감당하려면 몇 갈래로 나눠야 하나
- 그 갈래 수만큼 파티션·컨슈머를 미리 확보
- 컨슈머는 파티션 수를 넘을 수 없다 (상한)

시니어라면 : 느린 작업엔 방어를 건다

LLM은 느리다. 느린 컨슈머는 한 번에 조금씩, 넉넉한 시간으로 받아야 한다

배치는 작게, 시간은 넉넉히

- 한 번에 100명씩 (X) → 적게 받기
- 처리 제한 시간을 실측 지연보다 크게
- (한 번에 받는 양) × (건당 시간) < (제한 시간)

재시작에도 흔들리지 않게

- 배포·재시작 때 작업이 재분배되며 멈춤
- 신원 고정 + 점진적 재분배로 멈춤을 0에 가깝게
- lag이 쌓이면 컨슈머를 자동 증설

시니어라면 : 무손실·중복방지를 먼저 정한다

지연을 잡기 전에, 하나도 잃지 않고 두 번 처리하지 않을 것을 설계한다

실패를 정의한다

- 성공한 뒤에만 "처리 완료" 표시
- 실패 건은 버리지 말고 재처리 큐(DLQ)로
- 그래야 리밸런싱에도 중복·유실이 없다

비용·안정성 상한

- LLM 동시 호출에 상한을 건다 (429 방어)
- 저장·색인은 묶어서 (건별 호출 금지)
- 발행 측은 중복·유실 방지 설정을 켜다

파트 B

그때 한 것, 그리고 돌아가면 했어야 할 것

부하 테스트 설계 : 무릎을 찾는다

Little's Law는 raw(집계) 수치만 준다. 병렬을 늘린 채로 재면 처리율이 섞여 안 보인다

1단계 : 단일 파티션으로 μ 재기

파티션 1개 · 컨슈머 1개로 고정

→ 한 대의 순수 처리율 μ 를 먼저 측정
(scale out 변수 제거)

2단계 : 램프업으로 무릎 찾기

λ : 10 → 30 → 60 → 90 RPS

$W \propto 1/(1-\rho)$: ρ 가 1에 가까우면 지연 폭증

Locust로 13명 실사용자 × 10회 반복, 유저별 P50/P95/P99 개별 측정

3단계 : 필요 파티션 수 산출

목표 $\lambda \div$ 측정한 μ = 필요 컨슈머(파티션) 수

→ 이 숫자로 Kafka 파티션 · config 세팅

"몇 RPS부터 P95가 60초를 넘나?" = 한계점

목표 60인데 한계도 60이면 → 헤드룸 0 = 위험

한계 발견 : SLO를 넘겼다

60 RPS에서 P95 = 84초

- PMO이 준 SLO는 60초. 84초는 목표 초과 = 실패
- 여기서 "느리다"고 보고하면 끝. 하지만 어디가 밀리는지를 물어야 한다
- 병목 후보: LLM 호출? consumer 처리율? ES write? batch 크기?

그때 한 것 (1) : 병목을 특정했다

P95 84초. 평균이 아니라 분위·단계별로 쪼갬다

범인: TopicDecider 프롬프트의 라벨 key 오류

- 에러를 group-by로 분석 → 특정 라벨에서만 재시도·실패 반복
- 그 재시도가 꼬리(P95)를 끌어올림. 평균만 봤다면 못 잡았을 원인
- "느리다" → "이 단계의 이 오류가 P95 병목이다" 로 진술이 바뀐다

그때 한 것 (2) : Scale out은 최후의 수단

병목을 찾았다고 바로 파티션을 늘리지 않는다

먼저: 단일 효율 ($\mu \uparrow$)

- 비동기 처리 : LLM 대기 중 다른 일
- 증분 수집 : Gmail quota 98% 절감
- 프롬프트 오류 수정 → 재시도 제거

그 다음: Scale out ($c \uparrow$)

- 단일 파티션 μ 측정값으로 필요 파티션 수 = 목표 $\lambda \div \mu$ 산출
- Kafka 파티션 $\uparrow$ = 컨슈머 병렬 (M/M/c), 키 = user_id (순서 보장)
- 단, USL 주의 : 무한정 늘리면 리밸런싱 비용으로 역효과

한 대의 처리율(μ)부터 재고 올린다. 그래야 파티션이 몇 개 필요한지 계산된다

그때 한 것 (3) : P95는 잡았다

P95 84초 → 47초 (목표 60초 충족)

- 단일 효율 최적화(비동기 + 증분수집)만으로 SLO 안으로 진입
- 결과 진술: "빨라졌다" (X)
- "PM이 준 P95 60초 SLO를, 측정으로 검증하며 충족했다" (O)

돌아가면 했어야 할 것 : 계획 단계에서

지연(P95) 하나는 잡았다. 하지만 설계 단계에서 미리 정했어야 할 것들은 그때 놓쳤다

병렬을 못 채운 문제

- 파티션은 넉넉히 열어놓고
- 컨슈머 수를 거기 맞춰 못 늘림
- lag이 쌓여도 자동으로 안 늘어남

배치를 너무 크게 잡음

- 한 번에 100명씩 받아 처리
- 제한 시간 안에 못 끝내면
- 처음부터 다시 = 중복 처리

실패를 안 정해둠

- 실패한 건을 어디로 보낼지 미정
- 에러가 나도 조용히 넘어가 유실
- 재처리 경로(DLQ)가 비어 있음

비용·안정성을 뒤로 미룸

- LLM 동시 호출에 상한이 없음
- 저장을 건건이 (뭉어 보내지 않음)
- 메시지 중복·유실 방지 설정 없음

왜 못 잡았나 : 순서의 문제

기능 중심 시야

- "작동한다"에서 멈춤
- 느리면 "느리다"고 보고
- 처리 능력부터 늘리려 함

측정 중심 시야

- NFR을 $\lambda \cdot W$ 로 번역한다
- 램프업으로 무릎을 찾는다
- 단일 효율부터, 증설은 최후

도구(Claude Code)는 그때도 있었다. 없던 건 무엇을 측정할지, 무엇을 먼저 지킬지에 대한 질문

한 줄 회고

병렬은 준비했지만 실제로 그만큼 못 돌렸고, 한 번에 너무 많이 받아 제한 시간 안에 못 끝내면 통째로 재처리됐다. 실패·중복·비용은 아예 정해두지 않았다.

전부 코드 짜기 전에 정했어야 할 결정이다

- 파트 A의 설계는 지금은 안다. 그때는 몰라서 못 했다
- 요구사항(NFR)만 받고도 이 우선순위를 처음부터 정하는 것, 그게 설계자의 일

다시, 그 면접으로

네이버 Q11 회수

"성과는 냈는데 무엇을 측정할지 몰라 증명 못 했다"

- 좋은 답은 "무엇을 만들었다"가 아니다
- 어떤 NFR을 받았고 · 어떻게 지표로 번역했고 · 어떤 병목을 찾아 SLO를 충족했는가
- 그것이 설계자의 언어다

감사합니다

무엇을 측정할 것인가

참고: Queueing Theory · DDIA · Fundamentals of Data Engineering · Kafka Design Docs